Nicolas McDonald
CSCI580, Fall 2026

Assignment #4 Report

Problem Context:
To use the simulated annealing and genetic algorithms on a Traveling Salesman Problem (TSP) and compare and contrast the results through experimentation and the creation of graphs to show our findings and the shortest lengths tours that each algorithm produced.

There were 40 cities used in this TSP with the genetic algorithm set to 200 generations and the simulated annealing algorithm set to 20,000 iterations. The baseline heuristic used to run these algorithms was the nearest neighbor algorithm which just used the distance to its closest neighbor to create its tour.

An important part of the simulated annealing algorithm was the two-opt-swap algorithm shown here:

```
//////////////////////////////////////////////////////
def two_opt_swap(tour: Tour, i: int, k: int) -> Tour:
    # Return a new tour where the segment [i:k] is reversed.
    return tour[:i] + list(reversed(tour[i:k + 1])) + tour[k + 1:]

def random_two_opt_neighbor(tour: Tour, rng: random.Random) -> Tour:
    # Pick a random 2-opt move.
    n = len(tour)
    i = rng.randrange(0, n - 1)
    k = rng.randrange(i + 1, n)
    return two_opt_swap(tour, i, k)

//////////////////////////////////////////////////////
```

This part of the simulated annealing algorithm helped uncross edges and changed only two edges at a time so when it reversed the cities in a random part of the tour it changed the pathing only slightly. This slight variation when combined with simulated annealing's temperature cooling schedule leads to many solutions being explored at the start, but as the temperature cools down the algorithm is less likely to accept solutions that will make its tour length worse.

SA Parameters:
Iterations: 20,000
T_0: 0.2

alpha: 0.995
seed:123
Report every: 2000
Best Tour Length: 5.0057

An important part of the code for the genetic algorithm is the order_crossover_ox() function:

//

```
def order_crossover_ox(parent1: Tour, parent2: Tour, rng: random.Random) -> Tour:
    # TODO: order crossover (OX)
    n = len(parent1)
    reordered_list = [None] * n
    ctys_notin_range = []
    ctys_in_range = []
    count = 0

    i = rng.randrange(0, n - 1)
    k = rng.randrange(i + 1, n)

    reordered_list[i:k+1] = parent1[i:k+1]
    ctys_in_range = parent1[i:k+1]

    for c in parent2:
        if c not in ctys_in_range:
            ctys_notin_range.append(c)

    for i in range(n):
        if reordered_list[i] is None:
            reordered_list[i] = ctys_notin_range[count]
            count += 1

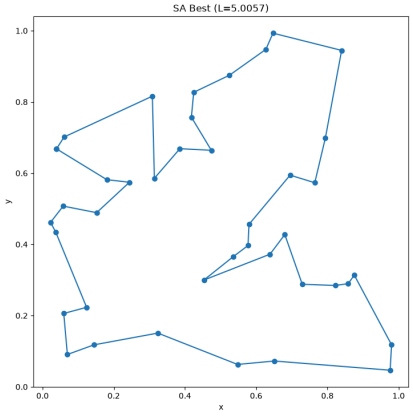
    return reordered_list
```

//

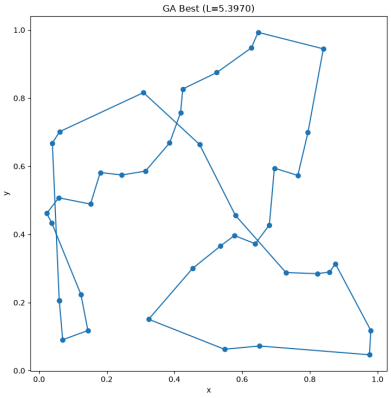
This code helps choose to combine the better tours together and switch their order to see if even better tours could be found by keeping a random range of cities in the tour of one parent in the same order, then filling in the rest of the cities with those not used in the second parent.

GA Parameters:
population size: 200
generations: 400
tournament k: 5
crossover rate: 0.9
mutation rate: 0.8
elite size: 5
seed: 999
report every: 50
Best Tour Length: 5.397

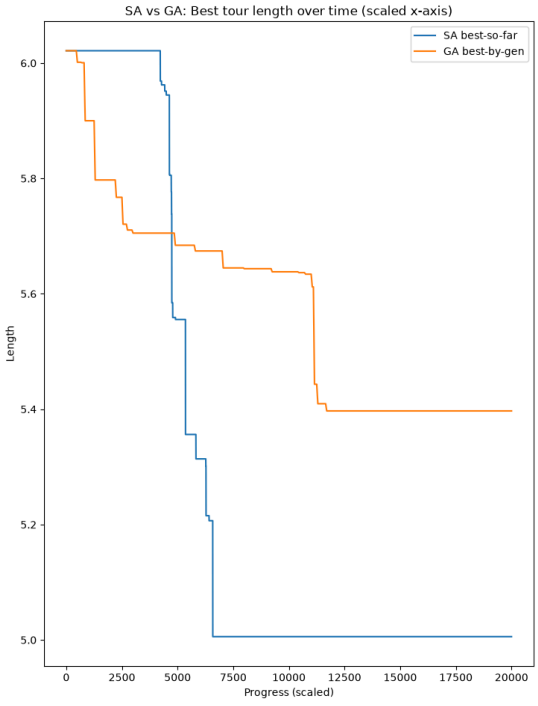
Comparison between GA vs SA:
When comparing how GA and SA improved upon the Nearest Neighbor algorithm, GA improved by 10.37% and SA improved by 16.87% based on my configurations. The reason why SA worked better than GA was because its two opt swap method was more efficient than GA's mutate swap method. This is the case because two opt swap only changes 2 edges at a time and makes sure to make sure that edges that cross each other become untangled as seen here in this graph:



The GA algorithm however changes four edges at a time because it is just swapping two cities which each have 2 edges each, ultimately changing four edges in total. This can drastically change how long it takes for the algorithm to find a smaller tour leading to needing a higher mutation rate and more generations to get shorter and shorter tours. GA shows crossing of edges here:



Question 1: Which algorithm converged faster?

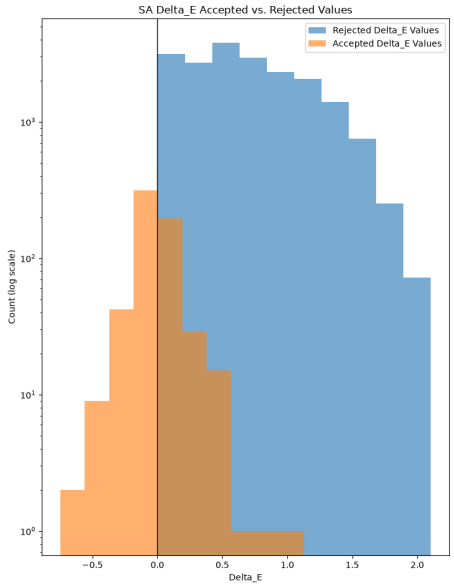


This graph shows that the simulated annealing plot (in blue) converged to a constant length of about 5.0 at around 6500 iterations. The genetic algorithm converged to a length of about 5.4 at around 11000 iterations, or when you take the scaling into consideration it is around 220

generations. Simulated annealing converged faster because it used a better method to try different paths of cities using the two-opt-swap which only changes two edges max leading to less variation which could lead the algorithm to explore paths that aren't as efficient. The genetic algorithm swaps two cities which means that 4 edges could have been changed, therefore having a greater chance of leading the algorithm astray.

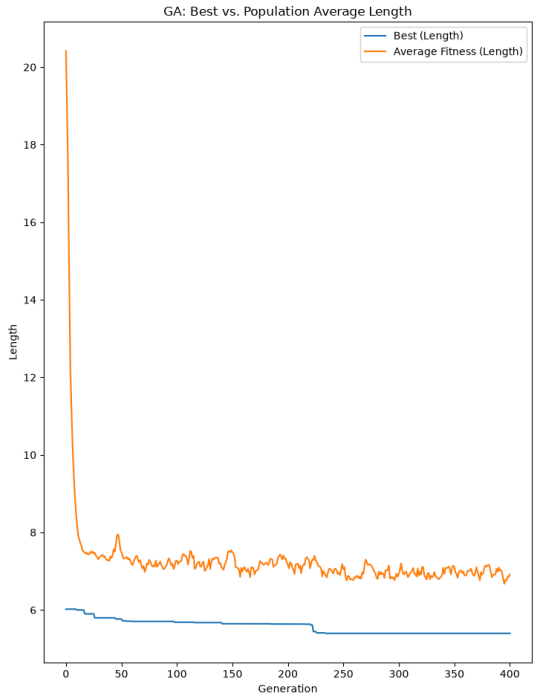
Question 2: Which algorithm showed more variability?

When running each algorithm multiple times they each didn't show much variability in the best lengths they were producing, although the simulated annealing algorithm consistently had the shorter tour length over all the runs. While experimenting with the genetic algorithms numbers to see how much shorter I could get it's best length to be I found that by using a mutation rate of 0.8 and a tournament_k value of 5 I was able to get the length down a little lower to 5.397.
Question 3: Did SA escape local minima?



This graph shows that SA escaped local minima because the longer tours that it accepted (to the right of the zero value) when it was at its local minima made it possible to find tours that were shorter. Without allowing a percentage of those longer tours to be accepted the algorithm wouldn't have escaped local minima because there would be no other tours to explore therefore being robbed of the ability to find shorter routes.

Question 4: Did GA lose diversity?



This graph shows that although the graph started with a high amount of diversity when it had length 20, within the first couple of generations on average (graph in orange) the lengths at each generation reached a point where they stayed within a tight range from 0.1-1 length. Therefore, the genetic algorithm lost its diversity very quickly.